M25 : Hardware for AI

Bitnet orchestration on RISC-V Multicore

Implemented Bitnet on RISC-V Multicore to efficiently accelerate AI hardware performance.

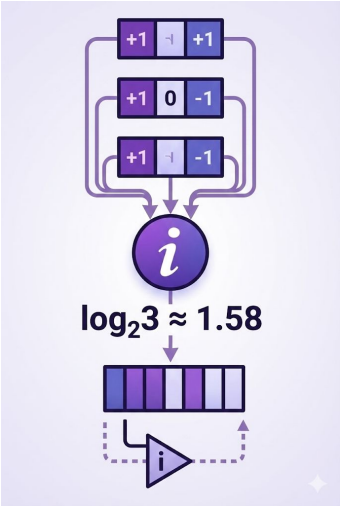
Team Name: Kernel Panic
Team Members: Priyanshi Jain
Pranav Shankar
Vedant Pahariya

About the Problem Statement

The Challenge: Single-core processors, even with the latest vector extensions, cannot sustain the massive data throughput and compute density required for LLM's General Matrix-Matrix Multiplication (MMM) operations.

Relevance: The project leverages the BitNet (1-bit LLM) model, which is incredibly efficient and memory-saving, replacing resource-intensive floating-point multiplications with simple addition/subtraction, therefore, making hardware acceleration highly effective.

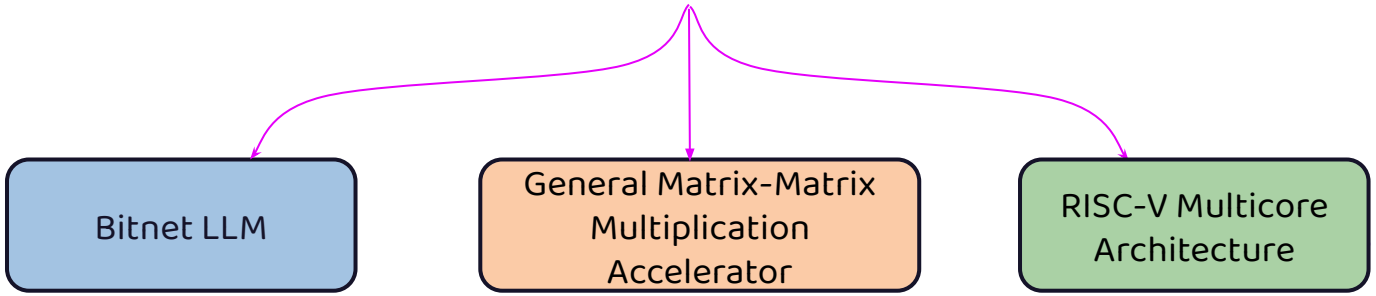
Gaps in existing approaches: Existing RISC-V heterogeneous systems lack a unified software stack, coherent memory flow and flexible hardware support- forcing manual, inefficient coordination between scalar, vector and accelerator units.



Solution

Implementing the BitNet (1-bit LLM) model across a **Multicore RISC-V** Architecture with custom MMM accelerator extension.

Complete full-stack architecture consisting of following three components:



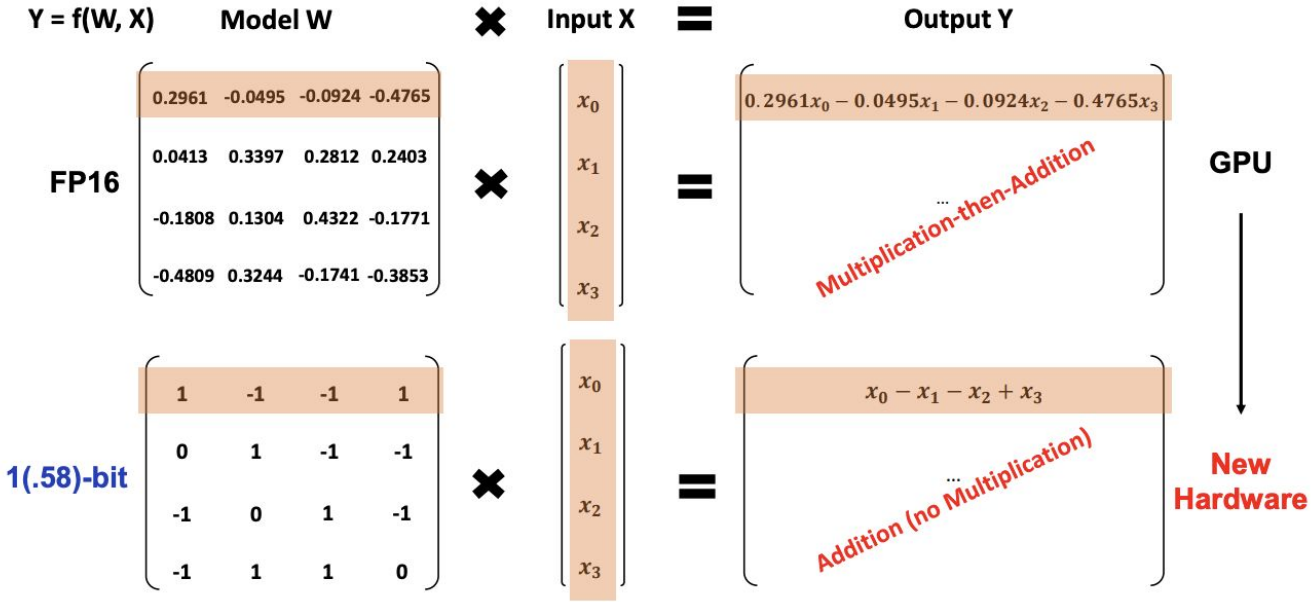
Why Bitnet b1.58?

- Incredibly efficient, memory-saving 1-Bit model
- Ternary Weights (just -1, 0, 1)
- All Computations are just simple Addition or Subtraction

Features	Traditional LLMs	1-bit LLM
Weight Precision	16/32 bit floating point	1 bit (Efficient)
Memory Footprint	Very large (7 billion ≈ 14/28 GB)	Very small (7 billion ≈ 0.8GB)
Operations	Floating point multiplication(which is resource intensive and power hungry)	**Simple Addition or Subtraction**

Why Bitnet b1.58?

- Incredibly efficient, memory-saving 1-Bit model
- Ternary Weights (just -1, 0, 1)
- All Computations are just simple Addition or Subtraction



How 1.58 came?

The real question arises how are we storing the weights as there is no such physical thing called 1.58 bits.

Say we have 5 weights to stored say W_1, W_2, W_3, W_4, W_5 we treat them as single 5-digit number in Base-3

$W_{seq} = (+1, -1, 0, +1, 0)$

After mapping it to Base-3 Digits, we get:

$D_{seq} = (2, 0, 1, 2, 1)$

$D = (20121)_3 = (178)_{10}$

So we will just store $(178)_{10}$ in binary form which is $(10110010)_2$

Total permutation = $3^5 = 243$

Maximum numbers that can be represented by 8 bits = $2^8 = 256$

Final efficiency in storing = 8 bits/5 weights = 1.6 bits/weight.

From Information Theory, minimum no. of bits for storing 3 weights is given by $\mathbf{Log_2 3 \approx 1.58}$

Paper explaining the storing of ternary weights and Bitnet 1.58(27 Feb 2024): Here

Why not other LLMs?

Models	Params	Bits	Size	Architecture
Bitnet	**2.4 B**	**1.6**	**1.1 GB**	**30 Layer Transformer**
Llama-2	7-70 B	16	13 GB	32 Layer Transformer
Mistral	7 B	16	14 GB	32 Layer Transformer
GPT-3.5	175 B	16	350-800 GB	96 Layer Transformer

BitNet Inference

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS python

```
generate: n_ctx = 2048, n_batch = 1, n_predict = 128, n_keep = 1

== Running in interactive mode. ==
 - Press Ctrl+C to interject at any time.
 - Press Return to return control to the AI.
 - If you want to submit another line, end your input with '\'.

System: You are a helpful assistant
> Hi
Hello! How can I assist you today?

> tell me about yourself
I'm an AI assistant designed to help you find information, answer questions, and provide support on a variety of topics. I can assist with research, provide explanations,
and offer guidance on many subjects. How can I help you today?

> What is Transformers in ML Models
Transformers are a type of machine learning model architecture, primarily used in natural language processing (NLP). They were introduced in 2017 by Vaswani et al. in the
paper
>
```

Target Architecture

Hardware: The core hardware is a RISC-V Multicore system (CVA6 CPU + ARA Vector Accelerator) with a custom General Matrix-Matrix Multiplication (MMM) accelerator extension (GEMMINI in our case)

Software: Scala, System Verilog (RTL Design), C/C++ for kernel and synchronization

Constraints:

- Interface incompatibility between CVA6 and Gemmini.
- Ara can only be simulated using licensed VCS on chipyard.

Chipyard is used at the end to make the complete SoC using Rocket, Saturn and Gemmini.

What did we do?

Phase I

-> Literature Review (Understanding Ara and CVA6 architecture)

-> Integrating CVA6 with Ara

Phase II

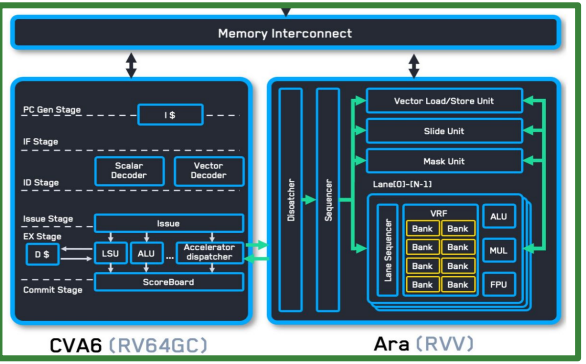
-> Implemented a Multicore system

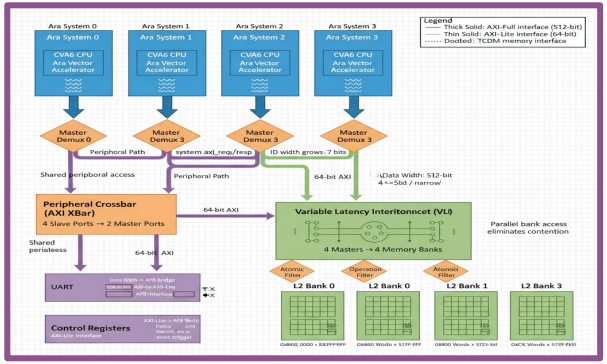
-> Designed the general Matrix-Matrix-Multiplication (MMM) accelerator

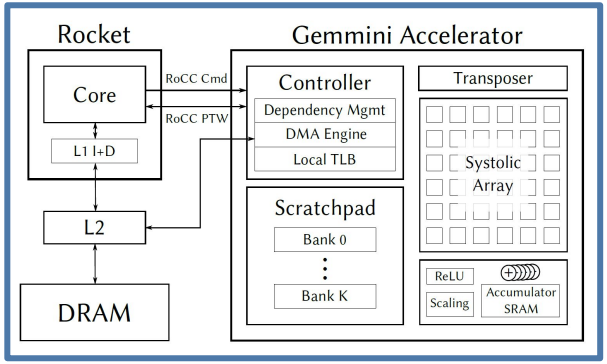
Phase III

-> Integrated Processing Unit with the accelerator

-> Chipyard





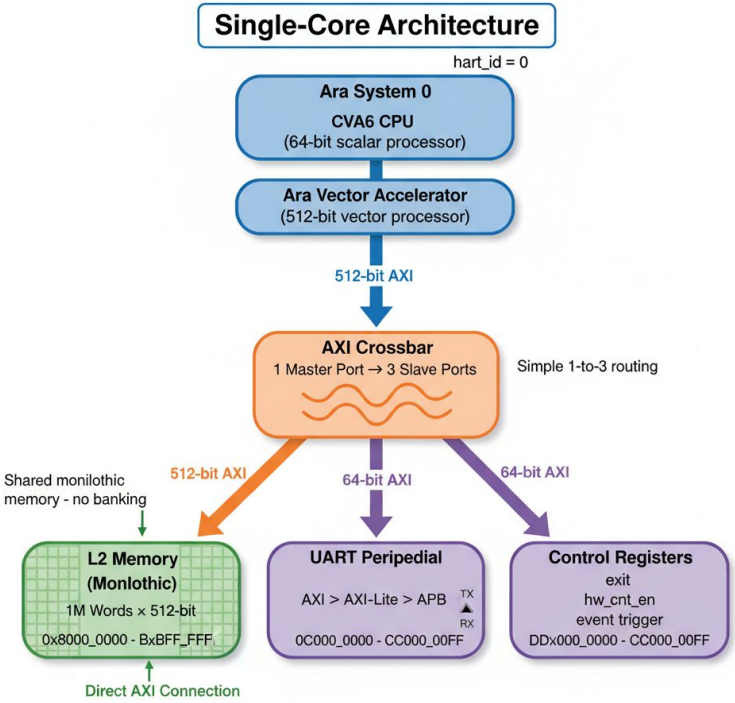


01

Vector Core

Going deep into the Implementation

Single Vector Core Architecture

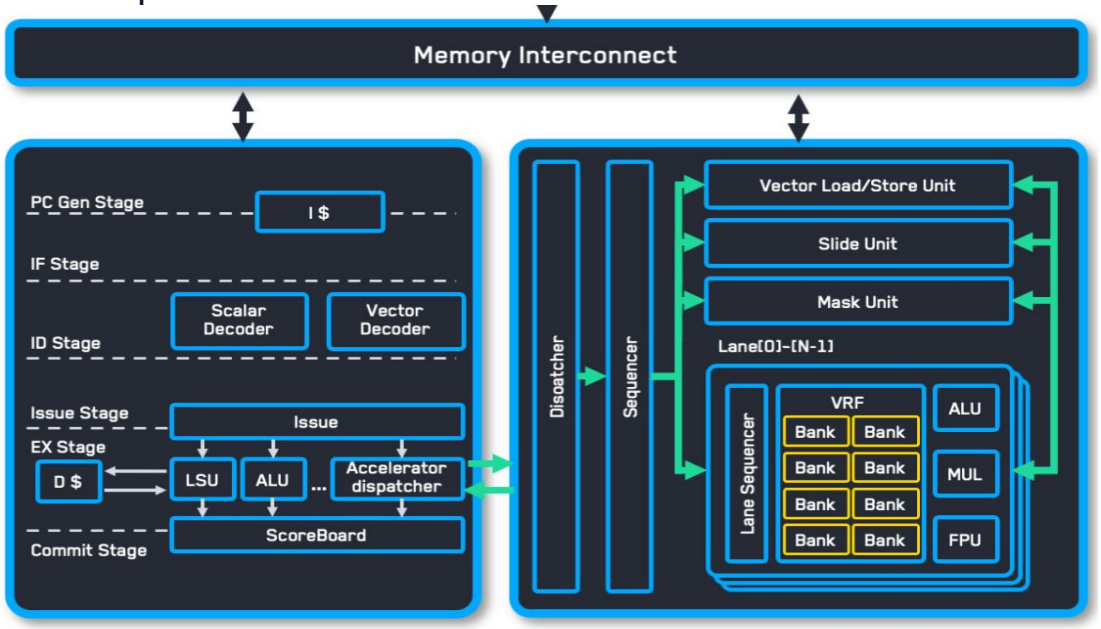


- One Processing Unit (1 CVA6 + 1 Ara Accelerator)
- Unified memory space (Single Large L2 Cache)
- Simple 1-to-3 Crossbar

```
├─ L2 Memory     (0x8000_0000)
├─ UART          (0xC000_0000)
└─ CTRL          (0xD000_0000)
```

Why we need Vector Core?

During LLM inference, Vector Core is required for handling the computation for Activation layers where we need Floating point Operation



CVA6 (RV64GC)

Ara (RVV)

Results on Vector Core

16 Lanes 16384 vlen

Matrix Size	Cycles	Performance (FLOP/cycle)	Utilization (%)
4x4	692	0.185	0.58
8x8	921	1.112	3.47
16x 16	2589	3.164	9.88
32x32	8028	8.163	25.51
64x64	33847	15.490	48.40
128x128	172772	24.277	75.86

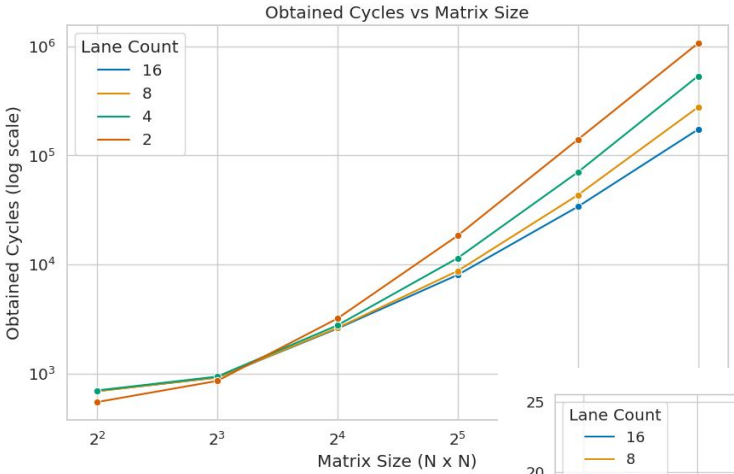
8 Lanes 8192 vlen

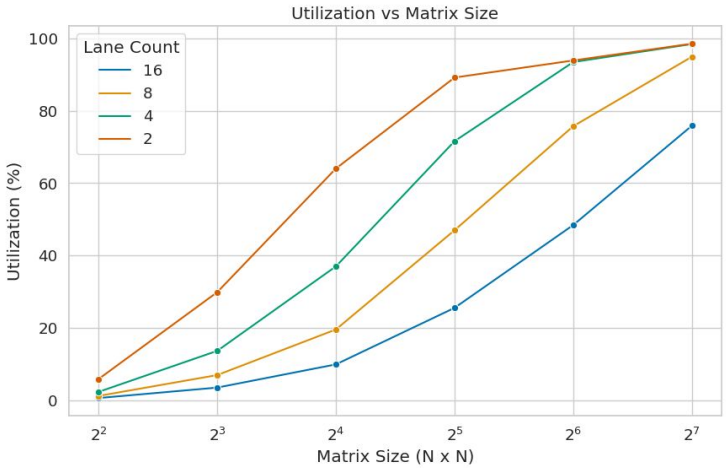
Matrix Size	Cycles	Performance (FLOP/cycle)	Utilization (%)
4x4	690	0.186	1.16
8x8	923	1.109	6.93
16x16	2622	3.124	19.53
32x32	8722	7.514	46.96
64x64	43273	12.116	75.72
128x128	276181	15.187	94.92

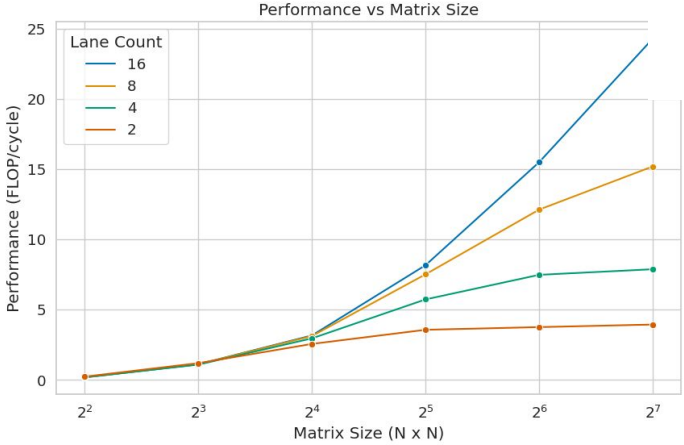
2 Lanes 2048 vlen

Matrix Size	Cycles	Performance (FLOP/cycle)	Utilization (%)
4x4	549	0.233	5.83
8x8	858	1.193	29.84
16x 16	3199	2.561	64.02
32x32	18374	3.567	89.17
64x64	139599	3.756	93.89
128x128	1064261	3.94	98.53

Visualization of Results





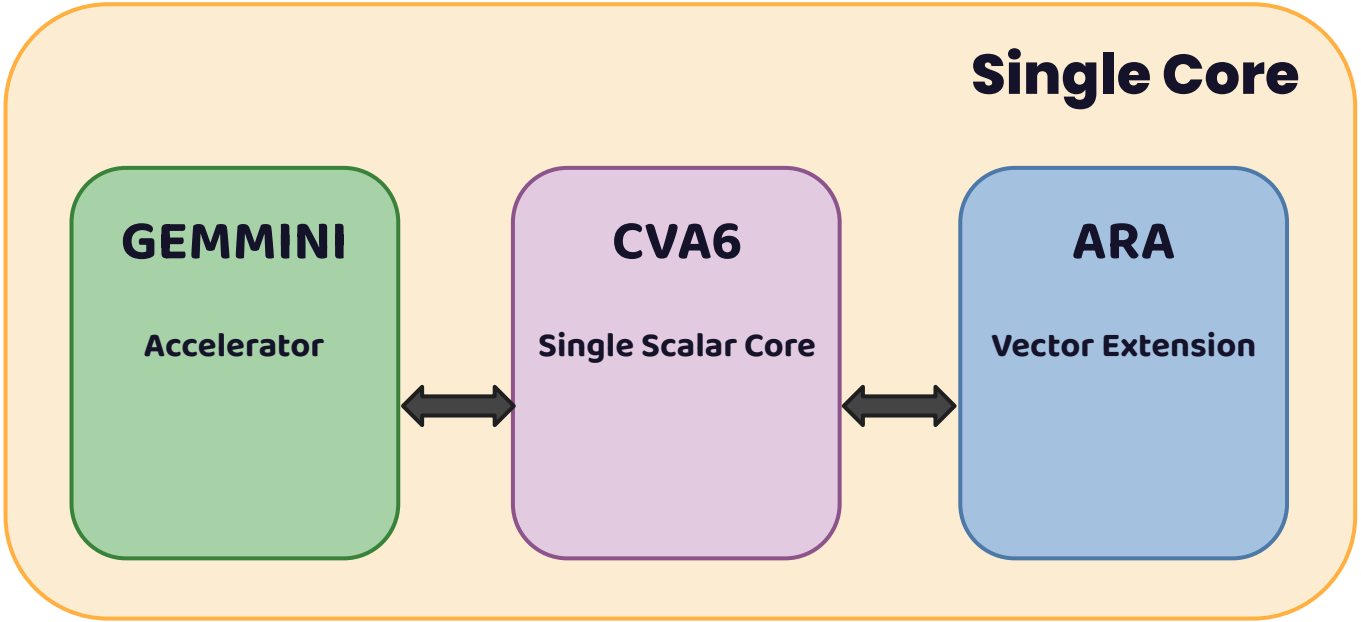


02

Multicore Architecture

Unleashing the full potential of RISC-V

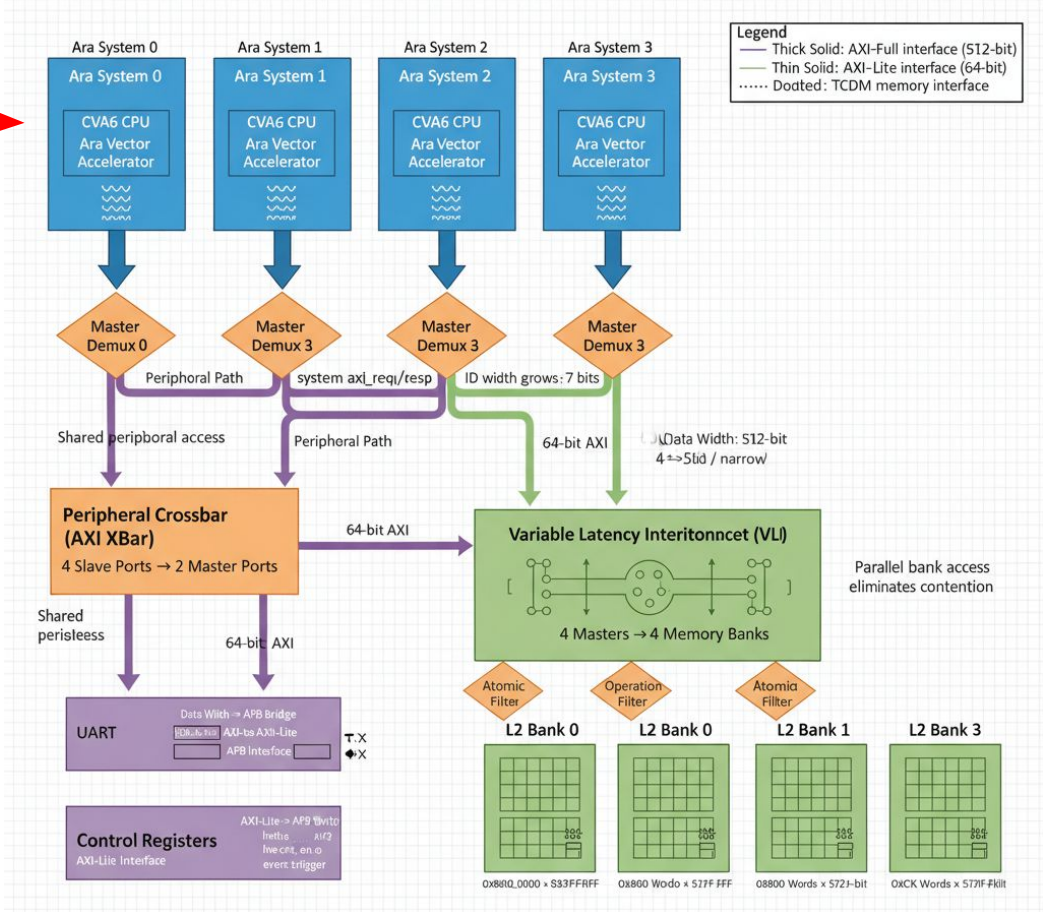
What makes up a Core?



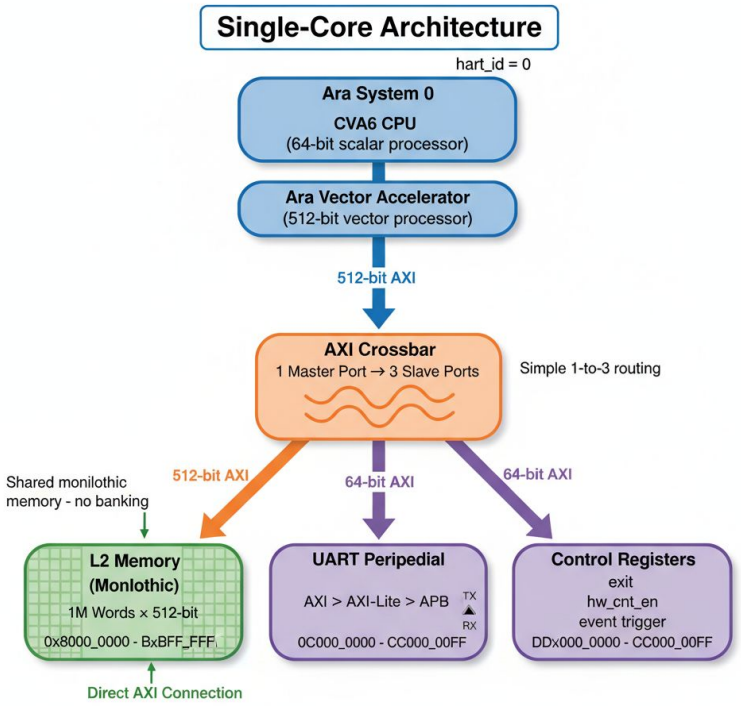
Accelerator-Vector Core

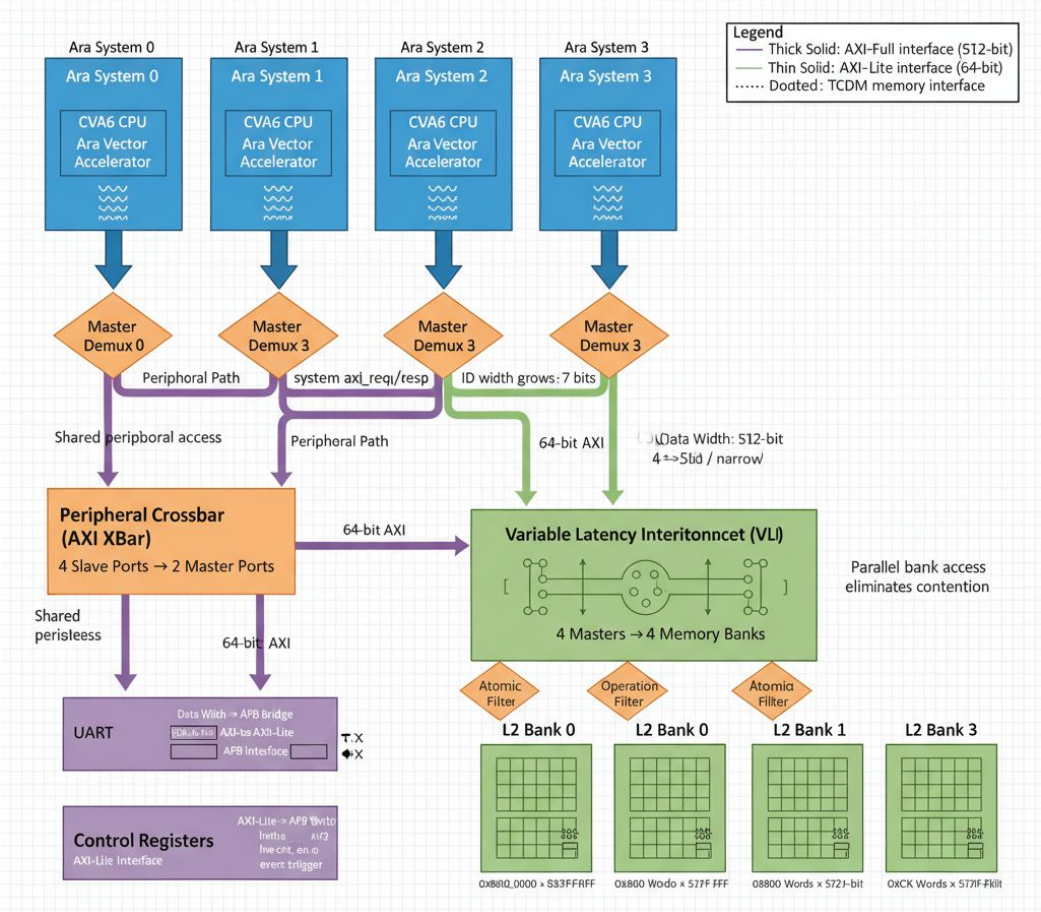
Multicore Architecture

Multi-Core analysis: We then extend our analysis to study how the vector core architecture behaves in a multi-core environment and explore its performance/efficiency trade-off. The system is modified to accommodate multiple clusters of the previously implemented system, and the SRAM main memory is multi-banked to support multiple Ara2 instances, with one bank per Ara2 and every bank with a parallelism of $4 \times L$ Byte each, with L being the number of lanes of each Ara2. Moreover, we implement a lightweight synchronization engine via system-level CSRs to let the clusters synchronize at the end of the execution. The main objective is to evaluate the impact of the multi-core



Single Core Vs Multicore





Results on hello_world Kernel

- This is basic Hello_World Compilation which shows successful the working of Multicore Architecture.
- In Multicore (4 Cores here), each letter is printed 4 times because of 4 cores
- Compile timing of both Single and Multicore is same thus showing Parallelism.

```
65  int main(int core_id) {  // ← Accept core_id as parameter
66    if (core_id == 0) {
67      printf("Ariane says Hello!\n");
68    }
69    primitive_synch(core_id); // Synchronize all cores at this point
70    return 0;
71  }
```

PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL

```
vedu.p@gnode028:/ssd_scratch/ara/hardware$ app=hello_world make simv
Simulation running, end by pressing CTRL-c.
Ariane says Hello!
[hw-cycles]:          0
[3324] -Info: ara_tb_verilator.sv:59: Assertion failed in TOP.ara_tb_verilator: Core Test *** SUCCESS *** (tohost = 0)
- /ssd_scratch/ara/hardware/tb/ara_tb_verilator.sv:62: Verilog $finish
Received $finish() from Verilog, shutting down simulation.

Simulation statistics
====================
Executed cycles:  1662
Wallclock time:   6.686 s
Simulation speed: 248.579 cycles/s (0.248579 kHz)
```

Single Core

```
65  int main(int core_id) {  // ← Accept core_id as parameter
66    printf("Ariane says Hello!");
67    primitive_synch(core_id); // Synchronize all cores at this point
68    return 0;
69  }
```

PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL

```
vedu.p@gnode028:/ssd_scratch/ara/hardware$ app=hello_world make simv
Simulation running, end by pressing CTRL-c.
AAAArrrriiiiaaaannnneeee    ssssaaaayyyyssss    HHHHeeeellllllllooooo!!!![hw-cycles]:          0
[3282] -Info: ara_tb_verilator.sv:59: Assertion failed in TOP.ara_tb_verilator: Core Test *** SUCCESS *** (tohost = 0)
- /ssd_scratch/ara/hardware/tb/ara_tb_verilator.sv:62: Verilog $finish
Received $finish() from Verilog, shutting down simulation.

Simulation statistics
====================
Executed cycles:  1641
Wallclock time:   6.584 s
Simulation speed: 249.241 cycles/s (0.249241 kHz)
```

Multicore

Performance on Matrix Multiplication

- No. of Operations in Matrix Multiplication = 2 * M * N * P
- Runtime for compilation comes from the verilator run.

- Performance = 2 * M * N * P / runtime (FLOPs per cycle)
- Theoretical Peak Performance = 2.0 × NR_LANES × NR_CORES (FLOPs per cycle)
  - Each vector FMA (Fused Multiply-Add) instruction performs 2 operations in 1 cycle.
    - 1 multiplication: a*b
    - 1 addition: +c
  - Example: `vfmacc.vf v0, f0, v16` computes v0 = v0 + (f0 × v16) (2 FLOPs per element)

- Utilization = (Actual Performance / Peak Performance) * 100
  = (Performance/ (2.0 * NR_LANES * NR_CORES)) * 100;

Single v/s Multicore Performance

1 Core & 4 Lanes

Lane Count	Matrix Size	Expected Cycles	Performance (FLOP/cycle)	Utilization (%)
4 Lanes 4096 vlen	4x4	702	0.182	2.28

1 Core & 2 Lanes

Lane Count	Matrix Size	Expected Cycles	Performance (FLOP/cycle)	Utilization (%)
2 Lanes 2048 vlen	4x4	549	0.233	5.83

4 Cores & 4 Lanes Each

```
Matrix C (computed output) (4x4):
  [0,0]=0.811822  [0,1]=0.986332  [0,2]=0.377351  [0,3]=0.611118
 [1,0]=0.799048  [1,1]=1.173323  [1,2]=0.636807  [1,3]=0.745088
 [2,0]=0.656613  [2,1]=1.015152  [2,2]=0.638512  [2,3]=0.923340
 [3,0]=0.505484  [3,1]=0.860082  [3,2]=0.693069  [3,3]=0.775094
PASSED!

=====================================
  RESULTS
=====================================

Eecution time: 570 cycles
Performance: 0.225 FLOP/cycle
Utilization: 0.70% of peak
 (Peak = 4 lanes × 4 cores × 2 ops/cycle = 32 FLOP/cycle)
```

4 Cores & 2 Lanes Each

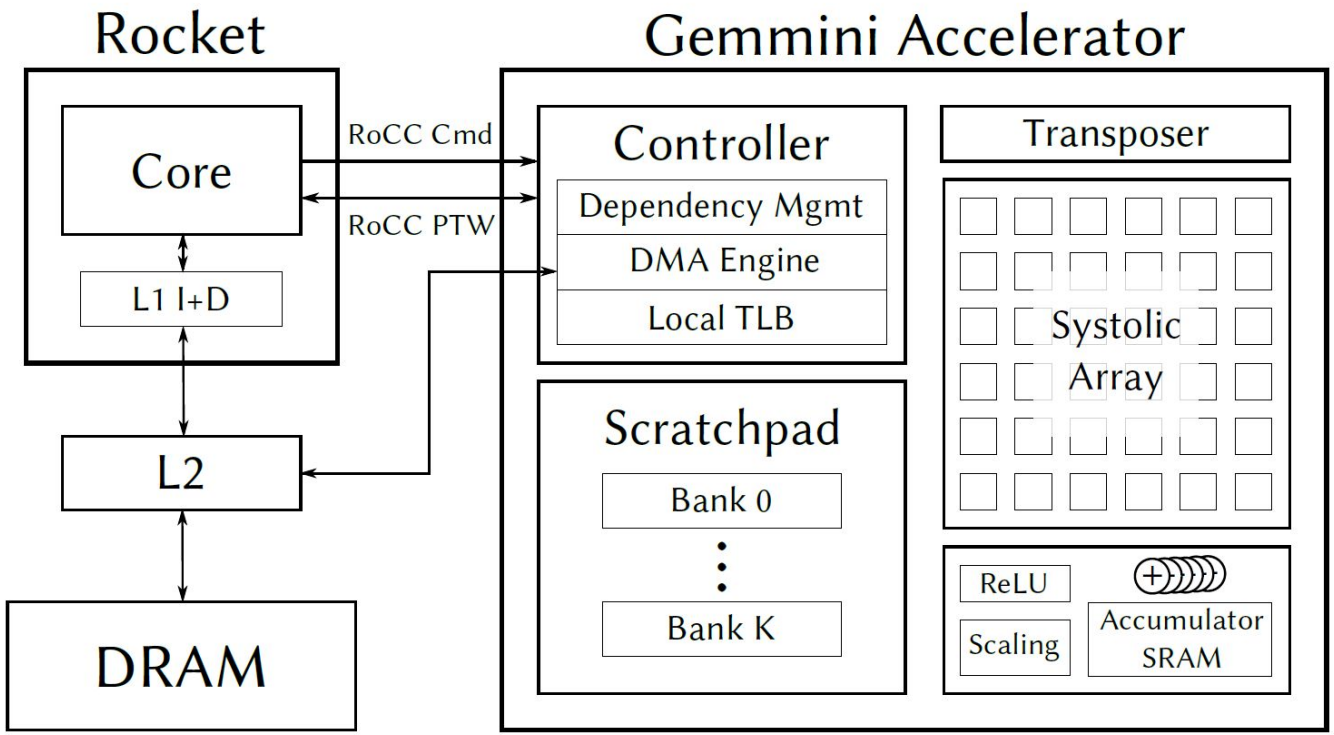
```
Matrix C (computed output) (4x4):
  [0,0]=1.202249  [0,1]=1.080332  [0,2]=1.014459  [0,3]=1.198725
  [1,0]=1.677711  [1,1]=1.414672  [1,2]=1.485581  [1,3]=1.035777
  [2,0]=1.325219  [2,1]=1.287282  [2,2]=1.363943  [2,3]=1.138194
  [3,0]=1.268400  [3,1]=0.946837  [3,2]=0.966652  [3,3]=1.065553
PASSED!

=====================================
  RESULTS
=====================================

Execution time: 463 cycles
Performance: 0.276 FLOP/cycle
Utilization: 1.73% of peak
  (Peak = 2 lanes × 4 cores × 2 ops/cycle = 16 FLOP/cycle)
```

03

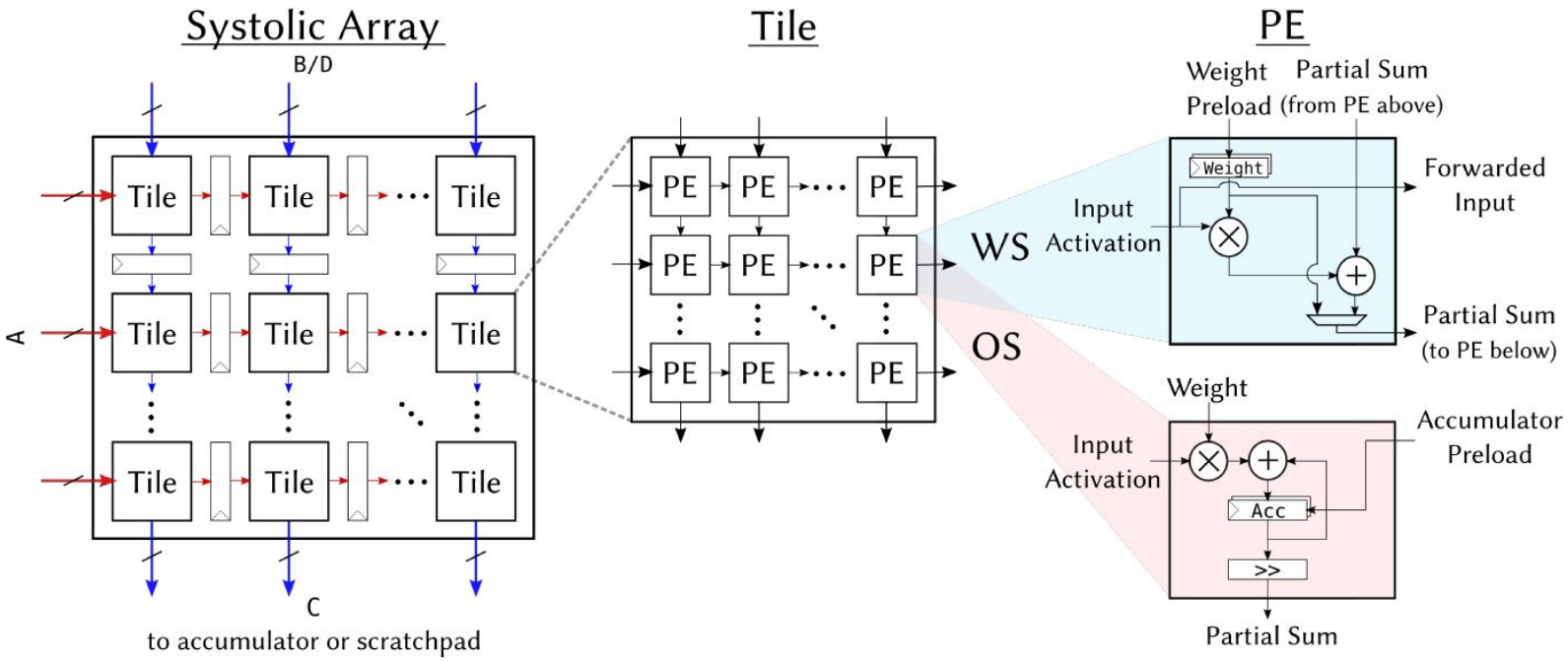
GEMM Accelerator

Going deep into the Implementation

Gemmini Accelerator

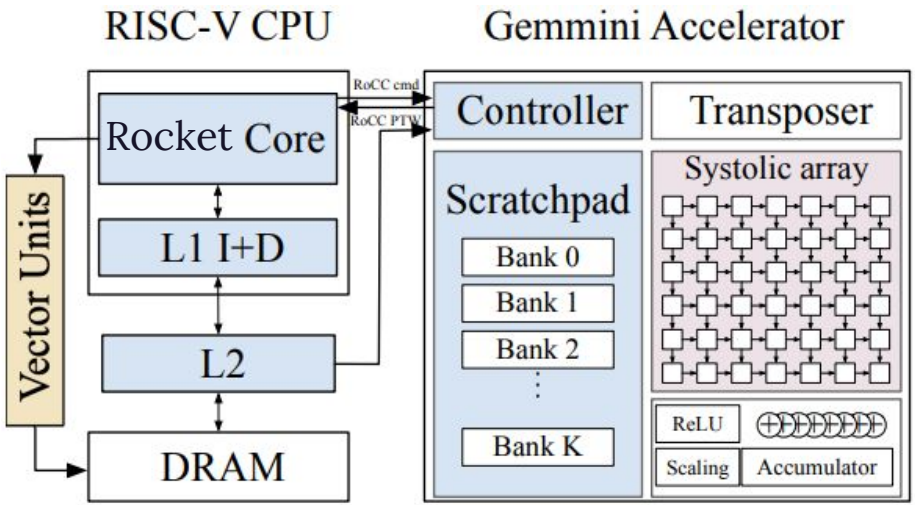


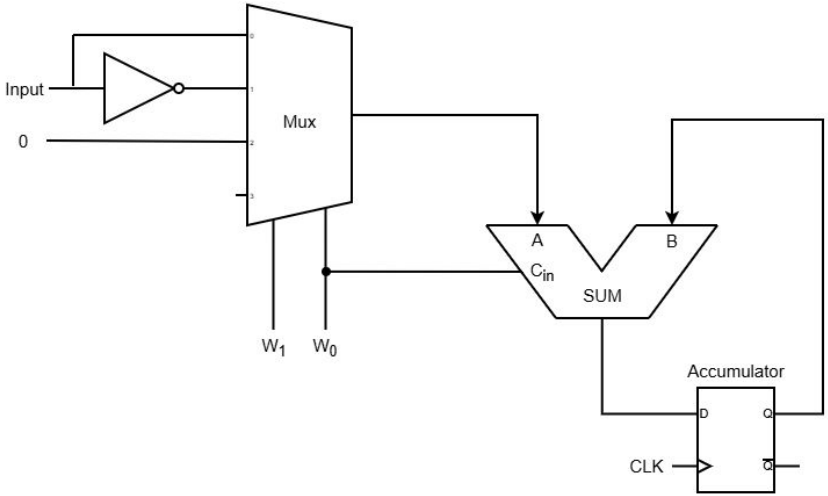
Gemmini Accelerator



Modified PE Structure

- Complete core design using chipyard
- Modified PE architecture





Modified PE Structure

```
class MacUnit[T <: Data](inputType: T, weightT
  import ev._
  val io = IO(new Bundle {
    val in_a  = Input(inputType)
    val in_b  = Input(weightType)
    val in_c  = Input(cType)
    val out_d = Output(dType)
  })

  io.out_d := io.in_c.mac(io.in_a, io.in_b)
}
```

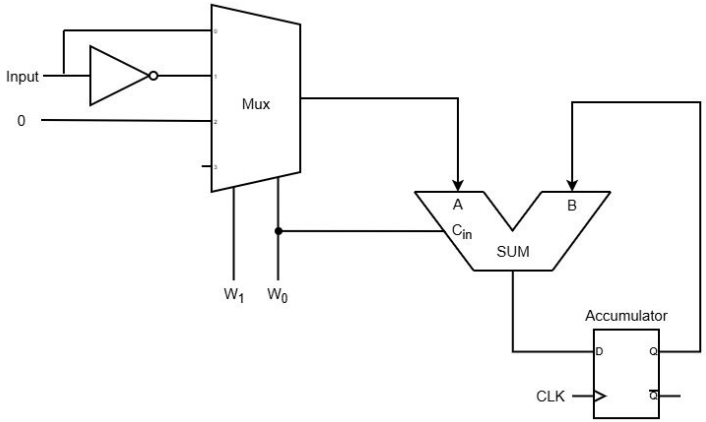
```
  if (has_ternary_weights) {
    // BitNet ternary weights: {-1, 0, 1}
    // Use bit patterns to identify ternary values:
    // 0 = 0x00, 1 = 0x01, -1 = 0xFF (int8 two's complement)
    val weight_bits = io.in_b.asTypeOf(UInt(8.W))
    val activation = io.in_a
    val accumulator = io.in_c

    // Check bit patterns for ternary values
    val is_plus_one = weight_bits === 1.U(8.W)
    val is_minus_one = weight_bits === 255.U(8.W)  // 0xFF = -1 in two's complement
    val is_zero = weight_bits === 0.U(8.W)

    // Compute result based on weight value
    val result = Wire(dType)
    when (is_plus_one) {
      result := (accumulator + activation).asTypeOf(dType)
    }.elsewhen (is_minus_one) {
      result := (accumulator - activation).asTypeOf(dType)
    }.otherwise {
      result := accumulator.asTypeOf(dType)
    }
    io.out_d := result
  } else {
    // Standard MAC operation for general weights
    io.out_d := io.in_c.mac(io.in_a, io.in_b)
  }
}
```

Gemmini performance Analysis

Dimension	Cycle Count	FLOPs/ cycle	Utilization (%)
64	349	1502.25	18.33
32	207	316.59	15.45
16	127	64.503	12.59

Novelty

- The complete redesign of the Gemmini Processing Element to be Ternary-Native
  - Reduces 60-80% Silicon area per PE
  - 16*16 array (256 PEs) could be replace by 32*32 (1024 PEs) with comparable area



- Zero-Skipping
  - While the MUX logic handles the correct mathematical output for zero (0), it still consumes dynamic power if the adder toggles.
  - Bitnet Sparsity: 0 (66%), 1 (17%), -1 (17%)
  - A logic gate checks the Weight bits ($W[1:0]$). If $W == 00$, the Clock Enable signal for the accumulator register and the input latch is de-asserted for that cycle

What did we learn or what felt *cool*?

Memory Mapping in multiple Lanes/Bank

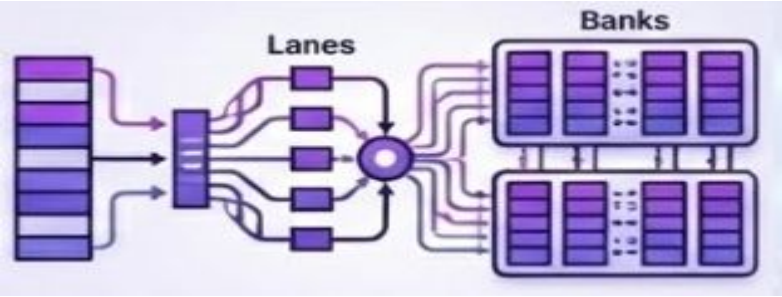
- How consecutive data are distributed across various Lanes and Banks and how they avoid consecutive data read conflict

Synchronization Mechanism:

- Lightweight synchronization using system-level CSR
- Important for maintaining the memory coherence

Storing weights in 1.58 bits

- $\log_2 3 \approx 1.58$ (Information Theory)



Thank You